12-Factor Agents - O estado da arte para os seus agentes

Em um mundo onde o desenvolvimento de software se transforma diariamente com o uso de AI, os sistemas desenvolvidos passam a ganhar uma nova dimensão com a visão de agentes. Agentes são capazes de através de uma interação com LLMs ir além de uma conversação podendo realizar ações, enriquecendo profundamente a experiência do usuário. Por volta de 2011 durante a onda de aplicações 'cloud native' surgiu um manifesto criado pela Heroku e intitulado "***The Twelve-Factor App***". Neste manifesto está descrito uma série de práticas a serem seguidas na construção das aplicações que rodam como serviços em nuvem, e suas 12 recomendações se encaixam para serviços agnósticos a linguagem e com qualquer combinação de serviços de apoio tais como bancos de dados, caches, mensagerias, entre outros.
Nesse momento onde AI vem redefinindo a forma como olhamos para o software presenciamos o surgimento de "***12-Factor Agents***". Inspirado em Twelve Factor App, 12-Factor Agents vem sendo popularizada por Dex Horthy (HumanLayer) após entrevistar mais de 100 fundadores e engenheiros construindo sistemas de AI em produção. Como resultado dessas entrevistas ele elencou 12 práticas que devemos considerar ao construir serviços como agentes.
Será demonstrado de forma prática como combinamos alguns pontos de arquitetura e Spring AI para implementar esses fatores.

1 - Linguagem natural para chamadas de ferramentas

A ideia aqui consiste na capacidade que os grandes modelos de linguagem têm em traduzir a linguagem natural para uma chamada de ferramenta estruturada.
Em sistemas tradicionais é comum a presença de IFs para determinar qual ação deve ser executada, ao utilizar LLMs é possível capturar a intenção do usuário e então o modelo decide qual ferramenta deve ser invocada.
Isso muda a forma como pensamos nas aplicações, ao invés de antecipar todo o fluxo possível, este passa a ser descrito como ferramentas que ficam disponíveis para os grandes modelos de linguagem decidirem quando e como invocá-las.

Exemplo @Tool

Com Spring AI temos algumas maneiras de resolver isso uma delas consiste no uso da annotations @Tool para assinar um método que poderá ser utilizado por um ChatClient:

```
@Tool(description = "Registrar entrada de estoque")  no
public String stockIn(String productId, int quantity){

    inventoryService.stockIn(productId,quantity);

    return "Entrada registrada";
}
```

Imagem 01 - Criando uma tool para ser executada pelo ChatClient

Após isso basta criar um @Bean com o chatClient:

```
@Bean   Eduardo Cesar *
ChatClient agentChatClient(ChatClient.Builder builder, InventoryTools inventoryTools){
    return builder
            .defaultTools(inventoryTools)
            .build();
}
```

Imagem 02 - Informando ao ChatClient qual Tool ele deve utilizar

2 - Seja dono dos seus prompts

Prompts é parte essencial para o sucesso de um bom agente, no entanto deve ser encarado como parte do design.
Quando construímos aplicações que interagem com LLM é importante compreender que temos alguns tipos de prompts. Para as instruções que são dadas ao agente esse tipo de prompt é conhecido como SystemPrompts, tais como seu comportamento, suas regras e ações.Outro tipo de prompt compreende a entrada do usuário aquilo que está sendo pedido ao LLM. Quando vamos descrever ações de uma ferramenta @Tool a descrição que adicionamos a ferramenta também é um prompt.
As instruções para os agentes podem mudar, um novo comportamento pode ser adicionado, um comportamento antigo removido, note a importância que é controlar esses itens afinal uma simples mudança, uma palavra alterada pode resultar em outro comportamento para o agente, então versione esses prompts e os externalize, não os mantenha de forma harded code.E por fim crie testes para os prompts mencionados, eles são parte do seu software.

Exemplo prompts

```
@Test
void envia_string_simples_sem_variaveis_ao_chat_client() {
    String prompt = "Você deseja registrar a ENTRADA de 10 unidades do produto 'camiseta'.";

    ChatClient chatClient = ChatClient.builder(chatModel).build();

    String resposta = chatClient.prompt() ChatClientRequestSpec
            .user(prompt)
            .call() CallResponseSpec
            .content();

    assertThat(resposta).isNotNull().isNotBlank();
}
```

Imagem 03 - Informando um prompt simples como string

Uma das opções ofertadas pelo Spring AI consiste em informar o valor como valor do prompt como uma String e passá-lo diretamente para o ChatClient.

Exemplo “PromptTemplate”

Uma outra possibilidade consiste em capturar valores dinâmicos do prompt do usuário, isso pode ser feito conforme mostrado na imagem abaixo:

```
@Test
void executa_prompt_renderizado_via_chat_client() {
    PromptTemplate template = new PromptTemplate(
            "Você deseja registrar a ENTRADA de {quantidade} unidades do produto '{produto}'."
    );
    String promptRenderizado = template.render(Map.of(
            k1: "produto",  v1: "camiseta",
            k2: "quantidade",  v2: 10
    ));

    ChatClient chatClient = ChatClient.builder(chatModel).build();

    String resposta = chatClient.prompt() ChatClientRequestSpec
            .user(promptRenderizado)
            .call() CallResponseSpec
            .content();

    assertThat(resposta).isNotNull().isNotBlank();
}
```

Imagem 04 - Trabalhando com inputs dinâmicos

Abaixo está o exemplo contendo tanto o prompt fornecido pelo usuário quanto o prompt do agente:

```
@Test
void envia_system_prompt_e_user_prompt_ao_chat_client() {
    ChatClient chatClient = ChatClient.builder(chatModel).build();

    String resposta = chatClient.prompt() ChatClientRequestSpec
            .system( text: "Você é um assistente de controle de estoque. Responda sempre de forma objetiva e confirme a operação solicitada.")
            .user( text: "Registre a entrada de 8 unidades do produto 'calça'.")
            .call() CallResponseSpec
            .content();

    assertThat(resposta).isNotNull().isNotBlank();
}
```

Imagem 05 - Exemplo construindo chatClient informando o prompt do sistema e o prompt vindo do usuário

3. Controle sua janela de contexto

Por padrão LLMs são stateless não armazenam qualquer resultado executado anteriormente, elas simplesmente não sabem qual prompt foi enviado, ou qual ferramenta foi executada anteriormente.
Esse entendimento está diretamente ligado ao que é chamado de contexto da conversa. Para que o LLM não “esqueça” o contexto é possível armazenar as mensagens que são trocadas ao longo de uma conversa. O ponto aqui não é referente apenas ao armazenamento mas também pertinente ao uso desses dados de conversa que passam a ser armazenados e enviados ao modelo, podendo aumentar drasticamente o consumo de tokens, principalmente ao se atentar que todos os prompts descritos, assim como ferramentas que foram descritas também entram nesta conta.
Com o propósito de controlar esse armazenamento e uso desses dados ao longo da interação com o modelo temos o que é conhecido como janela deslizante, que na prática permite o armazenamento desses dados por uma “janela” de mensagens, por exemplo é sempre enviado ao modelo as últimas dez mensagens trocadas. Vale ressaltar que também é possível utilizar mecanismos persistentes de longa duração como banco de dados para armazenamento.
O que o fator três busca garantir refere-se em manter o contexto de uma conversa ativo, sabendo quais ferramentas foram executadas, quais ações foram feitas pelo o usuário, permitindo que o modelo não se perca ao longo da interação, e o armazenamento persistente em banco de dados, tem grande valia para auditoria e rastreabilidade de qualquer interação realizada com o LLM.

Exemplo “inMemory”

Segue um exemplo persistindo o dado em memória por um ciclo de 10 mensagens:

```
@Test
void simula_conversa_com_historico_usando_memory_advisor() {
    String conversationId = "conversa-teste-001";

    ChatMemory memory = MessageWindowChatMemory.builder()
            .chatMemoryRepository(new InMemoryChatMemoryRepository())
            .maxMessages(10)
            .build();

    ChatClient client = ChatClient.builder(chatModel)
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(memory).build())
            .build();

    String resposta1 = client.prompt() ChatClientRequestSpec
            .user( text: "Quero registrar a entrada de 20 unidades do produto 'camiseta'.")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    assertThat(resposta1).isNotNull().isNotBlank();

    String resposta2 = client.prompt() ChatClientRequestSpec
            .user( text: "Quantas unidades eu disse que queria registrar, e para qual produto?")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    assertThat(resposta2).isNotNull().isNotBlank();
    assertThat(resposta2).containsIgnoringCase( sequence: "20");
    assertThat(resposta2).containsIgnoringCase( sequence: "camiseta");
}
```

Imagem 07 - Exemplo de uma conversa mantendo o histórico de mensagens

Exemplo “jdbcChatMemory”

Se estiver utilizando um banco de dados relacional pode preferir utilizar uma implementação baseada neste banco de dados. Neste contexto é possível utilizar JdbcChatMemory conforme exemplo mostrado abaixo:

```
@Test
void recupera_historico_do_postgres_em_chamadas_subsequentes() {
    String conversationId = UUID.randomUUID().toString();

    ChatMemory memory = MessageWindowChatMemory.builder()
            .chatMemoryRepository(jdbcChatMemoryRepository)
            .build();

    ChatClient client = ChatClient.builder(chatModel)
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(memory).build())
            .build();

    client.prompt() ChatClientRequestSpec
            .user( text: "Meu produto favorito é PRODUTO-XYZ.")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    String resposta = client.prompt() ChatClientRequestSpec
            .user( text: "Qual produto eu mencionei?")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    assertThat(resposta).containsIgnoringCase( sequence: "PRODUTO-XYZ");
}
```

Imagem 08 - Exemplo de uma conversa mantendo o histórico de mensagens utilizando JdbcMemory

4 - As ferramentas são apenas saídas estruturadas

As chamadas para ferramentas devem resultar em dados do tipo json que deve descrever o que um código determinístico deve executar, recebendo por exemplo os parâmetros de entrada como em uma API tradicional.
Da mesma forma que o prompt do usuário ao chegar no LLM ele deve resultar em uma saída json que descreve a intenção, e parâmetros, as execuções das ferramentas também devem resultar em um documento json estruturado contendo informações sobre o que foi executado, e qual retorno foi obtido. O LLM deve sempre emitir um JSON e nunca executar uma ação diretamente.
Logo a ideia consiste em ter uma forma de fornecer dados estruturados para que o modelo possa atuar e com isso minimizar os riscos de alucinação, tendo uma maior assertividade sobre o que é a intenção vinda do usuário e qual ferramento deve ser invocada, e uma melhor interpretação sobre o retorno da ação desta ferramenta.
Realizando isso com json temos uma padrão formalizado e popularmente conhecido e já utilizado em processos de integrações entre sistemas, mas agora fornecendo seus benefícios para um contexto de software como agente.

Exemplo json output structured

Abaixo é demonstrado como isso pode ser realizado na prática:

```
@Test
void llm_extrai_comando_stock_in_como_java_record() throws IOException {
    String systemPrompt = new ClassPathResource("prompts/system.prompt.st")
            .getContentAsString(StandardCharsets.UTF_8);

    ChatClient chatClient = ChatClient.builder(chatModel).build();

    InventoryCommand command = chatClient.prompt() ChatClientRequestSpec
            .system(systemPrompt)
            .user( text: "Registre a entrada de 10 unidades do produto 'camiseta'")
            .call() CallResponseSpec
            .entity(InventoryCommand.class);

    assertThat(command)
            .isNotNull()
            .satisfies( InventoryCommand cmd -> {
                assertThat(cmd.action()).isEqualTo(ActionType.STOCK_IN);
                assertThat(cmd.productId()).isEqualTo( expected: "camiseta");
                assertThat(cmd.quantity()).isEqualTo( expected: 10);
            });
}
```

Imagem 09 - Convertendo retorno do LLM em objetos

5. Unificar o estado de execução e o estado de negócios

Quando pensamos na interação com LLM o ponto crucial é o controle do contexto conforme mencionado no fator três. De forma direta é entendido que para o grande modelo de linguagem a única fonte de verdade é a própria conversa.
Ao pensar sobre o estado da execução ele pode ser entendido como um elemento que contém metadados sobre qual etapa o agente está executando “INICIANDO”, “AGUARDANDO”, “APROVADO”, ou sobre qual é o próximo passo, e isso é sobre o fluxo que o agente percorreu. Enquanto o estado de negócio refere-se sobre todo o ocorrido ao longo da conversa, quais foram as ferramentas chamadas, quantas vezes foram chamadas, resultados obtidos, note que isso é sobre todas as ações reallizadas.
Quando pensamos em um controle de contexto bem definido, lembre-se que separar o estado de execução do estado de negócio pode gerar uma complexidade desnecessária. Então sempre que possível mantenha os estados de execução e de negócios como partes ativas do contexto, persista-os seja por uma janela de mensagens, ou de forma duradoura, o importante é sempre permitir que o LLM consiga saber qual seu estado atual, se está aguardando uma aprovação, ou se acabou de chamar uma ferramenta para execução de alguma tarefa.De tal forma sempre que o LLM tiver acesso a todo o conteúdo de uma conversa uma vez que este esteja salvo o modelo consegue inferir as respostas sobre os estados mencionados anteriormente.

Exemplo unify execute state

Abaixo estão alguns exemplos de como isso pode ser implementado utilizando Spring AI.

```
@Test
void retoma_conversa_a_partir_do_historico_persistido() {

    String conversationId = UUID.randomUUID().toString();

    ChatMemory memory = MessageWindowChatMemory.builder()
            .chatMemoryRepository(jdbcChatMemoryRepository)
            .build();

    ChatClient client = ChatClient.builder(chatModel)
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(memory).build())
            .build();

    client.prompt() ChatClientRequestSpec
            .user( text: "Meu produto favorito é camiseta. Guarde essa informação.")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    ChatMemory memoriaRecuperada = MessageWindowChatMemory.builder()
            .chatMemoryRepository(jdbcChatMemoryRepository)
            .build();

    ChatClient clientRecuperado = ChatClient.builder(chatModel)
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(memoriaRecuperada).build())
            .build();

    String resposta = clientRecuperado.prompt() ChatClientRequestSpec
            .user( text: "Qual é o meu produto favorito?")
            .advisors( AdvisorSpec a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call() CallResponseSpec
            .content();

    assertThat(resposta) AbstractStringAssert<capture of ?>
            .isNotNull() capture of ?
            .containsIgnoringCase( sequence: "camiseta");
}
```

Imagem 10 - Única fonte de verdade para o LLM é a própria conversa

6. Iniciar/Pausar/Retomar com APIs simples

Bons agentes devem ser capazes de serem iniciados, pausados, e retomados de forma simples. Por exemplo, ao acessar o endpoint inventory/agent deve ser possível iniciar uma interação com o agente a partir dele.
Para todo o caso o agente já iniciado deve ser capaz de ser retomado do ponto de onde parou, e ser executado deste ponto até a sua conclusão. Os resultados das chamadas devem estar disponíveis para serem acessados por exemplo via web hooks, ou até mesmo enviados como um evento. Quando um agente está em “pausa” é de extrema importância que todo o seu estado até o momento seja persistido, conforme mencionado no fator 5.
Todo o fluxo de conversação costuma possuir um “conversation ID” que nada mais é que um identificador único para uma determinada conversa, similar a um ID já utilizado em banco de dados para identificar coisas de forma única, este mesmo pensamento se aplica aqui, mas neste caso sendo o ponto chave para recuperar toda uma conversa e com isso permitir que o modelo possa ser pausado ou interrompido e em seguida retomado do ponto que estava.
Esse é um ponto puramente arquitetural, expor através de uma api, disponibilizar os resultados em um endpoint de resposta que recebe como parâmetro um “conversation ID”, ou ainda publicar esses resultados como evento em uma fila, é o que deve ser feito neste ponto do agente. Não é apenas delegar ao LLM, é utilizar engenharia de software voltada para a construção de agentes.

7. Contacte humanos através de chamadas de ferramentas

Permitir que o LLM execute ações é interessante mas possui um certo grau de risco quando seguido por essa abordagem. O ideal é que seu agente tenha ferramentas que possam permitir ou restringir acesso a operações mais críticas e que deveriam ser avaliadas por um humano.
Agentes são indivíduos autônomos realizadores de tarefas, e isso pode incluir permitir que ele controle seu email, crie ou remova arquivos em seu sistema operacional, ou ainda no exemplo de um controle de estoque o agente pode criar ou remover algum item, tabela ou simplesmente decidir que deve excluir o próprio banco de dados.
A ideia por trás deste fator é justamente não permitir que o agente realize todas as tarefas da maneira que ele compreender e isso envolve disponibilizar ferramentas para que o usuário final possa decidir se deseja prosseguir ou não com uma determinada ação.
Vale notar a relevância de outros fatores atrelados aqui, isso tem relação direta com o fator 3, pois ter total controle sobre o contexto irá permitir ao modelo “se lembrar” de qual ação ele pediu aprovação.
O fator 4 sobre saídas estruturadas também ganha uma conexão a este item, pois é através das saídas estruturadas que o modelo sabe o que precisa ser feito, trazendo neste caso maior determinismo e previsibilidade nas ações do agente. Então exponha uma ferramenta “request_human_input” que terá como saída um json contendo os campos “question”, “context” e “urgency”, e com isso mantenha as saída do modelo estruturadas.
Além disso, o fator 5 que descreve a valia da unificação dos estados de negócio e execução isso simplifica o controle de estado do agente e permite que o modelo saiba exatamente tudo o que aconteceu até o presente momento.
Seguindo o fator 6 também se mostra presente neste item, pois o agente uma vez iniciado deve ser capaz de ser retomado exatamente do seu último ponto de execução, ou seja se o modelo realizou uma chamada para uma ferramenta de aprovação para cadastrar um produto, não faz sentido algum que após aprovação ser respondida por exemplo via webhook que a ferramenta executada neste momento seja uma ferramenta de favoritar produtos.
Portanto crie ferramentas @Tools para solicitar a interação dos humanos, o conceito atrelado a isso é Human in the Loop. Exponha uma ferramenta que permita ao modelo saber qual a decisão tomar, qual ferramenta ele deve executar e com qual urgência isso deve ser feito, sempre mantendo as saídas de interação com o modelo estruturadas.

8. Assuma o controle do seu fluxo de trabalho

O LLM sabe da intenção daquilo que ele acredita que precisa ser executado, mas o que será executado de fato deve estar controlado de forma determinística, e explícita. O modelo declara intenções e o código realiza as ações.
Pensando em uma chamada para o LLM que “peça a quantidade do produto camiseta”, neste caso o retorno do modelo pode declarar que é necessário executar alguma @Tool de busca, e então retornar imediatamente este resultado.
Outros tipo de ações como por exemplo a “remoção de 50 camisetas”, deve ser uma ação que irá interromper o loop para solicitar uma aprovação de usuário, neste ponto não importa qual seja o canal de aprovação, o importante é que ao realizar essa ação, o agente então recarregue todo seu contexto e siga informando sua intenção no uso da ferramenta para remoção, que por sua vez irá chamar o código responsável por essa ação. Todo esse ciclo está sendo controlado, e é rastreável, desde a solicitação, aprovação, e execução do código. Aqui o fator 5 com os estados unificados tem uma forte presença neste item, assim como o fator 6, pois está relacionado a forma como o agente pode ser exposto, e invocado, permitindo que o loop seja pausado e retomado de seu ponto específico.
Em resumo, o LLM não deve executar nada sozinho, todo o determinismo e previsibilidade devem ser de responsabilidade do desenvolvedor, do código construído e não simplesmente delegada ao modelo.

9. Compactar erros na janela de contexto

Lidar com erros ocorridos durante o processamento de uma ação sempre foi algo feito durante o desenvolvimento de software. Enquanto humanos, quando é identificado uma situação de erro, todo o log é analisado, as buscas por mensagens de erros começam a surgir, e eventuais soluções começam a ganhar vida através de N tentativas de modificação que almejam uma solução, com os grandes modelos de linguagem não deve ser diferente. No sistema que está controlando a entrada e saída de produtos, o modelo pode receber um prompt de usuário pedindo a remoção de um produto inexistente, ou ainda solicitando uma ação que não esteja no escopo deste agente, observe que ambos os casos podem levar a erros no comportamento do agente.
O ponto aqui está em resumir esse erro durante a interação com o LLM e quando um erro ocorrer ao invés dele simplesmente ser exibido como stack trace em algum log, ele deve ser inserido à janela de contexto da conversa. Isso deve permitir o grande modelo de linguagem analisar, e até mesmo propor ou executar ações de correção com base no erro apresentado.
É importante controlar a exibição de erros assim como já é feito ao aplicar logs em partes críticas do sistema, é levantado um entendimento sobre o que é necessário logar, e sobre o que realmente faz sentido e pode ajudar durante um debug. No contexto de agentes essa mesma preocupação continua valendo, devolver informações desnecessárias ou pouco estruturadas para o modelo não terá grande valia, além de resultar em um contexto inflado, e consumidor de tokens. Este pensamento está intimamente relacionado ao fator 3.
Assim como os humanos precisam analisar com detalhes um erro, para entender o acontecido e corrigir, para os grandes modelos de linguagem, erros são informação, e informação útil pertence ao contexto, não ao log, pois este serve de contexto para o humano.

10. Agentes pequenos e focados

No mundo da engenharia de software já é sabido o quão importante é compreender o domínio que está envolvido, as responsabilidades que são necessárias e como organizamos tudo isso de forma que faça sentido. Delimitar contexto como descrito em Domain Driven Design, ou como descrito por Robert Martin em Clean Architecture a letra "S" dos princípios SOLID que diz que "Uma classe deve ter um e, apenas um motivo para mudança", são um análogo que podemos fazer com relação ao fator 10.
Não crie um grande agente que "faça tudo", ao contrário disso compreenda o domínio que está atuando, delimite os pontos de atuação e integração com o LLM e por fim crie agentes focados em resolver um problema de forma específica.
No exemplo mencionado o agente que realiza o controle de estoque, não deve ser o mesmo agente que estará realizando a venda por exemplo na página principal do site, isso torna o contexto mal delimitado, as responsabilidade se confundem e e a ideia de um agente pequeno e focado acaba se perdendo.

11. Acione de qualquer lugar, encontre os usuários onde eles estão

Este ponto é puramente engenharia de software aplicada, é de conhecimento de todo desenvolvedor de software as possibilidades existentes para integrações entre sistemas, por exemplo o uso do protocolo HTTP que permite que sistemas troquem informações utilizando o modelo REST. Ao realizar um aprofundamento sobre o fator 11 que o agente deve ser capaz de ser acionado de qualquer lugar a reflexão mais tangível neste ponto é como permito que meu agente se integre a outros sistemas e agentes, e mais que isso que o fluxo conversacional faça sentido ao longo dessa trajetória.
O agente pode ser iniciado com uma conversa de WhatsApp, mas ele pode seguir por um fluxo onde envia um email de confirmação para o usuário, que ao visualizar e confirmar o email pode retomar a conversa no WhatsApp exatamente na etapa atual e prosseguir com a interação.
Vale notar o quanto esses fatores vão se relacionando, implementar o fator 6 seu agente passar a ter as características necessárias para que o fator 11 seja atingido de forma relativamente fácil.

12. Faça do seu agente um redutor sem estado

Vale notar que este fator 12 ele é uma conclusão sincera sobre o uso e aplicabilidade dos fatores mencionados anteriormente. Quando respeitada a implementação de fatores como 3,5, 6, e 8 naturalmente ele caminha para o décimo segundo fator, tornando seu agente um redutor de contexto sem estado de forma orgânica.